Year: 2026-2027 **Sem V-NEP**

Course: Agile Software Development & DevOps Lab

Date of Performance: 28/08/2026 **Date of Submission: 01/09/2026**

Class: T1

Student Roll No: 2401036 **Student Name: Dhruv Hemdev**

EXPERIMENT – 6

AIM : To study and Implement Agile Software Project lifecycle using Jira with DevOps Pipeline Integration.

LO MAPPED : LO1.

LO1 : Understand and apply Agile software development methodologies and project tracking tools to plan, manage and track the software project lifecycle.

THEORY :

Agile Software Development

Agile is an approach to software development in which work is carried out in short repeated cycles instead of one long phase. Requirements are broken into small pieces that can be delivered independently, and working software is produced at the end of every cycle. Because the product is reviewed after each cycle, a change in requirement can be accepted at any stage rather than only at the start of the project.

Agile Software Project Lifecycle

The Agile lifecycle repeats the same phases for every iteration. Requirements are collected and recorded in a backlog, a subset is selected during sprint planning, the selected items are developed

and tested, the increment is reviewed, and the work is released. Items not selected remain in the backlog for a later iteration, so the project advances through a series of small completed increments.

Scrum

Scrum is the Agile framework used in this experiment. Work is organised into fixed-length iterations called sprints. The product backlog holds all known requirements, and the items chosen for the current iteration form the sprint backlog. Progress is tracked on a board during the sprint, and the sprint is formally closed when the selected items are complete.

Scrum Board

A Scrum board shows the issues of the active sprint arranged in columns, one column for each state of the workflow. The board used in this experiment has the columns To Do, In Progress, In Review and Done. Moving a card from one column to the next records the transition against the issue, so the board always shows the current state of the sprint.

User Stories and Story Points

A user story describes a requirement from the point of view of the person who will use the software, and is small enough to be finished within a single sprint. A task is technical work that does not describe user-facing behaviour, such as configuring a build server. Story points are a relative estimate of the effort a story needs, and adding the points of the selected stories gives the planned size of the sprint.

Jira

Jira is a project tracking tool used to manage Agile projects. Each requirement is recorded as an issue with a unique issue key, a type, an assignee, an estimate and a status. Issues are shown on the board and moved through the workflow as work proceeds. When a sprint is completed, Jira produces reports from the recorded data.

DevOps

DevOps is a set of practices that brings development and operations together so that software can be built, tested and released frequently and reliably. It depends on automation, so that steps which would otherwise be carried out by hand are performed by tools in the same order every time.

Continuous Integration and Continuous Deployment

Continuous Integration is the practice of adding code changes to a shared repository regularly, where each change is automatically built and checked so that errors are found early. Continuous Deployment extends this by automatically delivering the code that passes those checks to the target environment.

Git and GitHub

Git is a distributed version control system that records the history of changes to the source code and allows each feature to be developed on its own branch. GitHub is a remote hosting service for Git repositories, and it is the repository from which the automation server obtains the application files and the pipeline definition.

Jenkins

Jenkins is an open-source automation server that performs the stages of the pipeline. A Declarative Pipeline defines those stages in a file named Jenkinsfile which is stored in the repository with the application source code, so the pipeline is versioned along with the code. Jenkins can also be configured to start a build by itself when the repository changes, which removes the need to press Build Now.

Poll SCM

Poll SCM is a Jenkins trigger that checks the repository at a fixed interval and starts a build when a new commit is found. It achieves the same result as a GitHub webhook, and is used here because Jenkins is running on localhost and cannot receive an incoming webhook from GitHub.

Docker

Docker packages an application together with the environment it needs into an image, which can then be run as a container on any machine that has Docker installed. Because the image contains the web server as well as the application files, the application behaves the same way wherever the container is run.

Integration of Jira with the DevOps Pipeline

Jira and the pipeline are linked through the issue key. Each branch is named after the issue it implements and each commit message begins with the same issue key, so the code change that

satisfies a requirement can be identified from the Jira issue and the build that carried it. The team also agreed a definition of done which requires the automated build to succeed before an issue may be moved to Done, so the state of the board reflects the state of the pipeline.

Agile and DevOps Lifecycle Flow:

Requirement recorded in Jira → Story estimated and pulled into the sprint → Branch created and code committed with the issue key → Pull request reviewed and merged → Jenkins detects the change and runs the pipeline → Application containerised and deployed → Issue verified and moved to Done → Sprint completed and reports generated

EXECUTION :

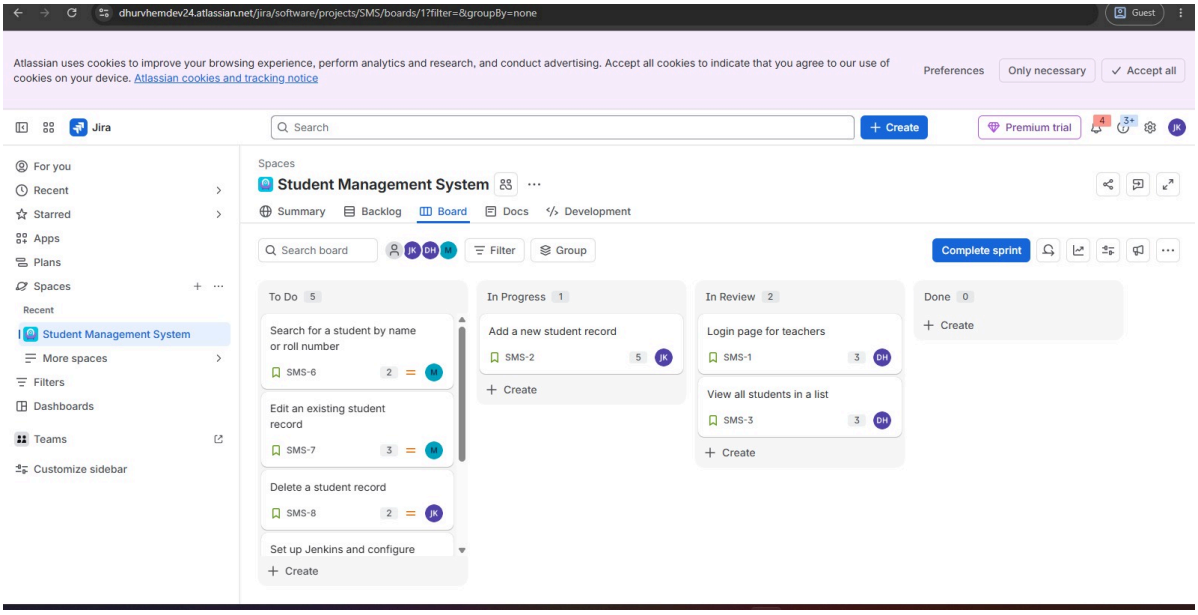
The application used throughout this experiment is the Student Management System, a static web application consisting of index.html, app.js and style.css. It was planned in Jira, version controlled in the GitHub repository dhruvh6/student-management-system, built automatically by Jenkins and deployed in a Docker container.

Step 1: Create a Jira Account and a Scrum Project

A free Atlassian account was created and a new software project was added, choosing the Scrum project template so that the work could be organised into sprints. The project was named Student Management System and was given the key SMS, which is the prefix of every issue key in the project.

Step 2: Understand the Scrum Board

The Scrum board of the project was examined. The board shows the issues of the active sprint in columns, one for each workflow state. Cards are dragged from one column to the next as work progresses, and the count above each column shows how many issues are currently in that state. The Complete sprint button at the top right is used to close the sprint once the selected issues are finished.



Step 3: Create User Stories

Each requirement of the application was raised using Create Issue. Requirements describing behaviour visible to the teacher using the application were raised as Stories, and technical work such as configuring the build server and containerising the application was raised as Tasks. Every issue was assigned to a team member and the stories were estimated in story points. The six user stories of the sprint total eighteen points.

The 6 user stories — 18 points

Key	Story	Points	Owner
SMS-1	Login page for teachers	3	Dhruv
SMS-2	Add a new student record	5	Janhavi
SMS-3	View all students in a list	3	Dhruv
SMS-6	Search for a student by name or roll number	2	Mayank
SMS-7	Edit an existing student record	3	Mayank
SMS-8	Delete a student record	2	Janhavi
	Total	**18**	

Split per person: Dhruv 6, Janhavi 7, Mayank 5.

The complete sprint backlog, including the two technical tasks, is listed below.

Table 1: Sprint Backlog of Project SMS

Issue Key	Type	Summary	Points	Owner
SMS-1	Story	Login page for teachers	3	Dhruv
SMS-2	Story	Add a new student record	5	Janhavi
SMS-3	Story	View all students in a list	3	Dhruv
SMS-6	Story	Search for a student by name or roll number	2	Mayank
SMS-7	Story	Edit an existing student record	3	Mayank
SMS-8	Story	Delete a student record	2	Janhavi
SMS-9	Task	Set up Jenkins and configure automatic build	—	Janhavi
SMS-10	Task	Containerise the app with Docker and run the container	—	Mayank
		Total story points	18	

The eighteen points were divided between the three members as six for Dhruv, seven for Janhavi and five for Mayank, which kept the load of the sprint roughly even.

Step 4: Create and Start the Sprint

A sprint named SMS Sprint 1 was created and all the issues of the backlog were dragged into it. Starting the sprint moved the selected issues onto the board and began the iteration, after which the board became the working view for the team.

Step 5: Create a GitHub Account and Repository

A GitHub repository named student-management-system was created to hold the application source code. The repository was initially empty and the project files were pushed into it in Step 8.

Step 6: Install Git

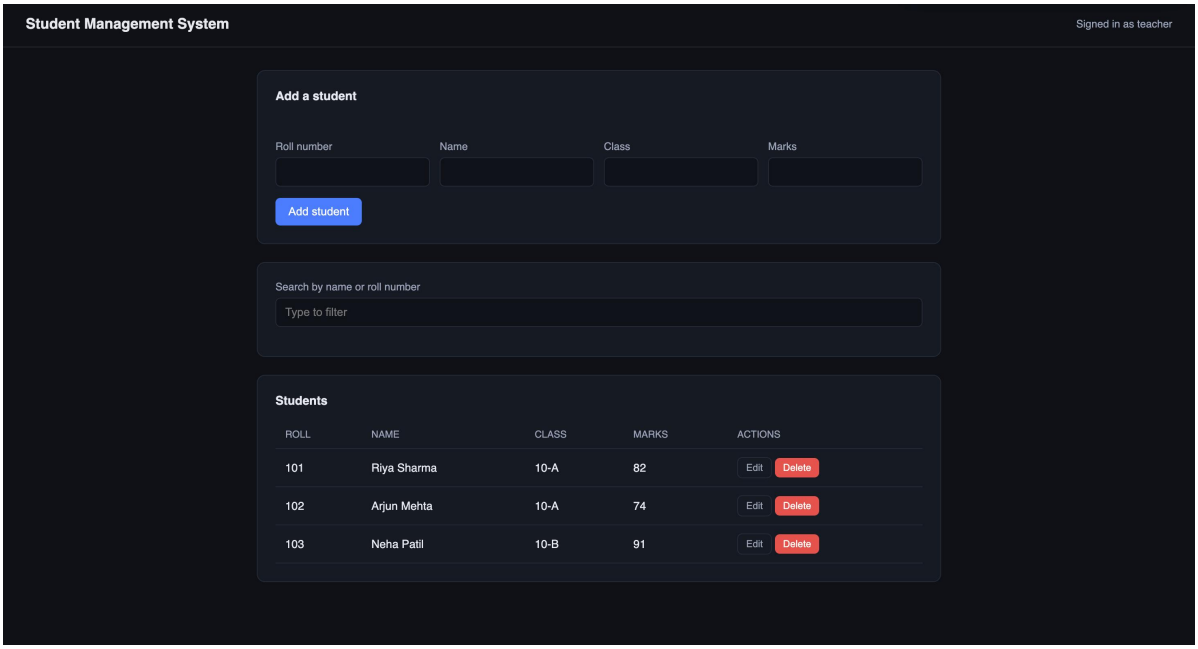
Git was installed on the development machine and the installation was verified from the terminal.

```
git --version
```

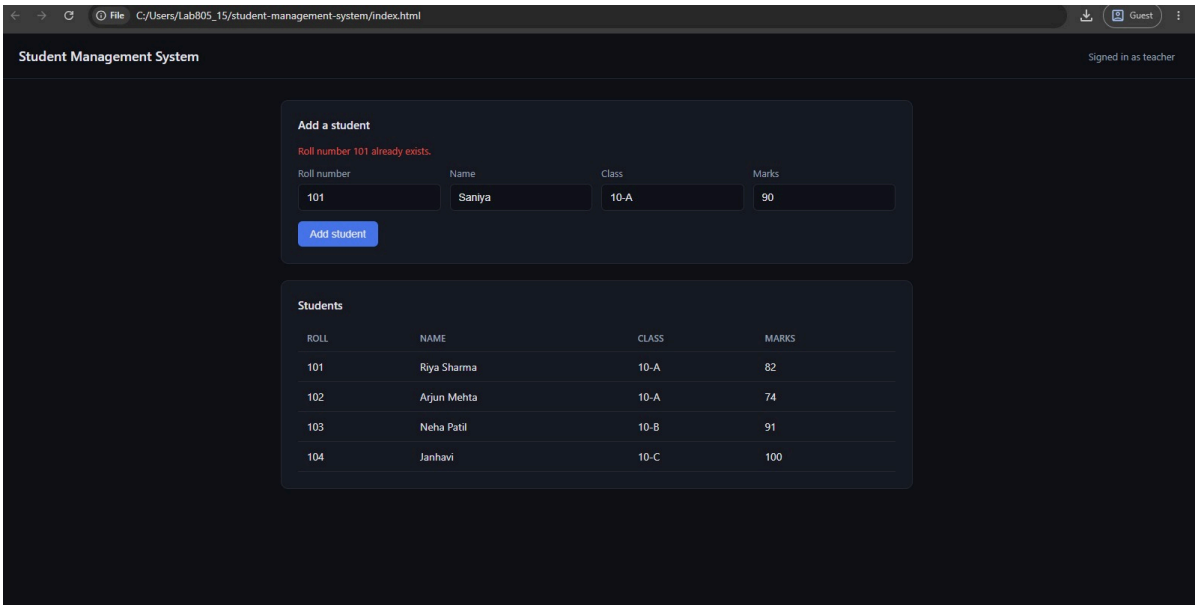
Step 7: Create the Application

The Student Management System was written as a static web application using HTML, CSS and JavaScript, with the records kept in the browser using localStorage so that no backend or database is required. The interface provides a teacher login, a form to add a student, a search box and a

table of students with Edit and Delete actions, which together implement the six user stories of Table 1.



The application also validates the data entered by the teacher. Attempting to add a student with a roll number that already exists is rejected with a message instead of creating a duplicate record.



Step 8: Upload the Code to GitHub

The project folder was initialised as a Git repository, the files were staged and committed, the GitHub repository was added as the remote named origin, and the code was pushed. Each commit message begins with the key of the Jira issue it implements, which is what links the repository history back to the board.

```
git init
git add .
git commit -m "SMS-1 Login Page Completed"
git remote add origin https://github.com/dhruvh6/student-management-system.git
git push origin main
```

Every story was developed on its own branch named after its issue key, for example feature/SMS-8-delete, and merged into main through a pull request reviewed by another team member.

[

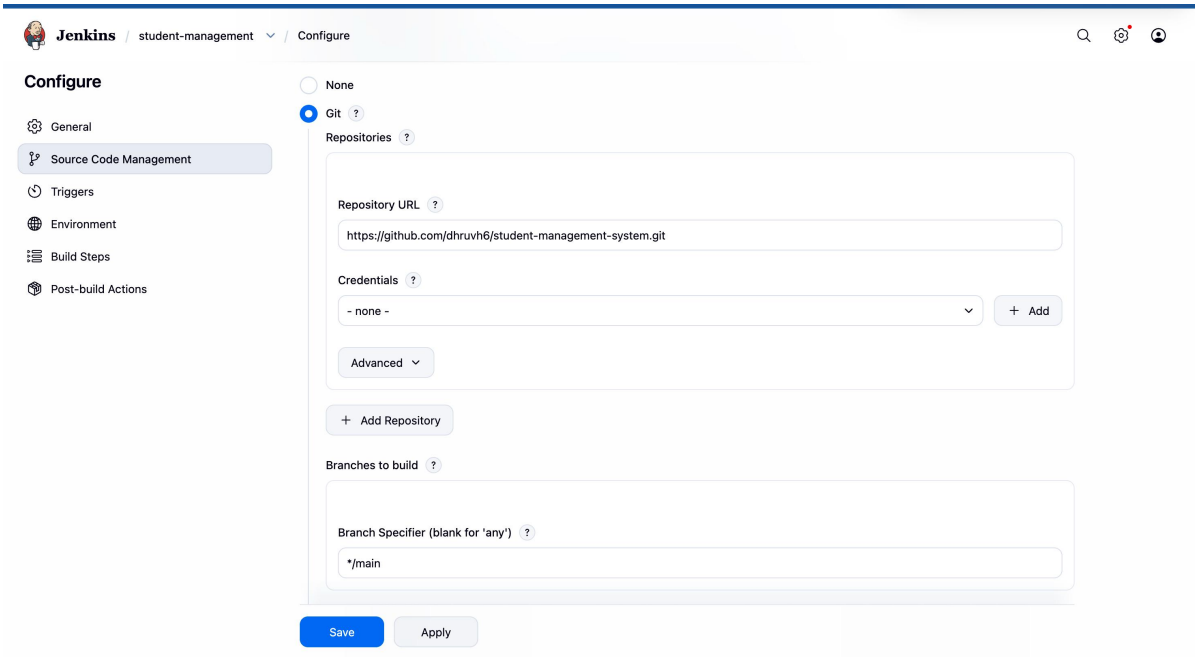
Step 9: Install Jenkins

Jenkins was installed and started, and the setup wizard was completed by unlocking Jenkins with the generated administrator password, installing the suggested plugins and creating an admin user. The Jenkins dashboard is served at port 8080 of the local machine.

```
http://localhost:8080
```

Step 10: Connect Jenkins with GitHub

A Jenkins job named student-management was created and its Source Code Management section was set to Git. The URL of the GitHub repository was entered and the branch to build was set to main, so that Jenkins fetches the application from GitHub rather than from a local folder.



Step 11: Build the Project

The job was executed with Build Now. Jenkins cloned the repository, checked out the latest revision of main and ran the stages defined in the Jenkinsfile committed with the source code. The pipeline verifies that the required files are present and then builds the Docker image.

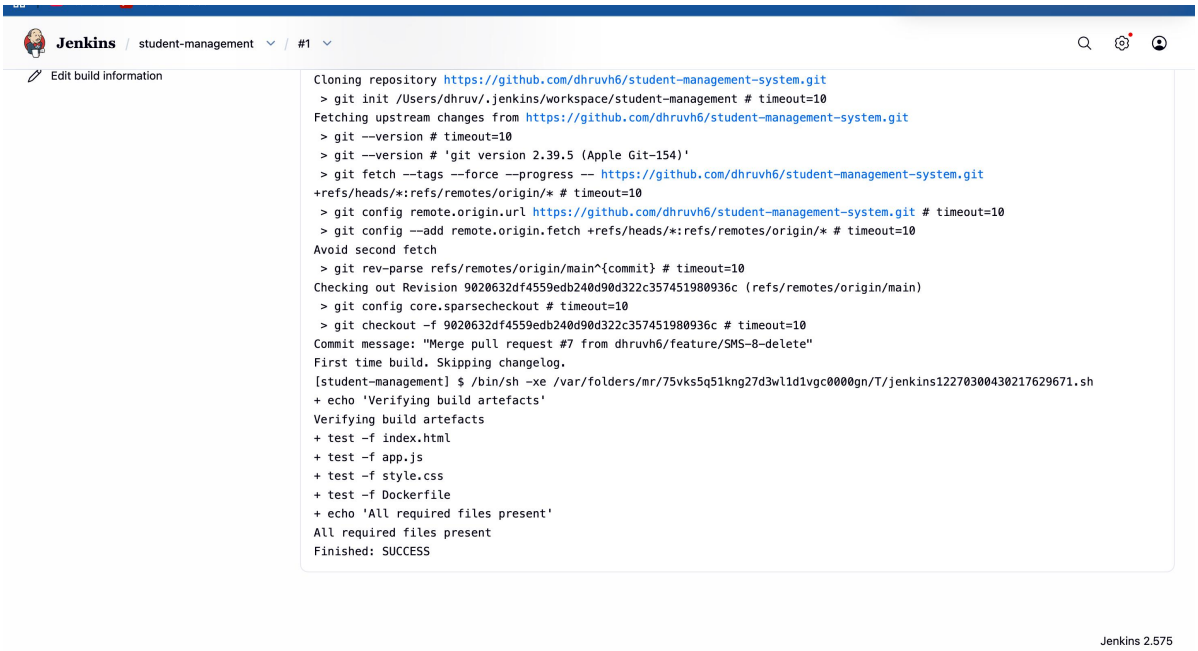
```
pipeline {
    agent any

    stages {
        stage('Checkout') {
            steps {
                echo 'Fetching source from GitHub'
                checkout scm
            }
        }
        stage('Verify') {
            steps {
                echo 'Checking that required files are present'
                sh 'test -f index.html'
                sh 'test -f app.js'
                sh 'test -f Dockerfile'
            }
        }
        stage('Build Docker image') {
            steps {
                echo 'Building the container image'
                sh 'docker build -t student-management .'
            }
        }
```

```
    }

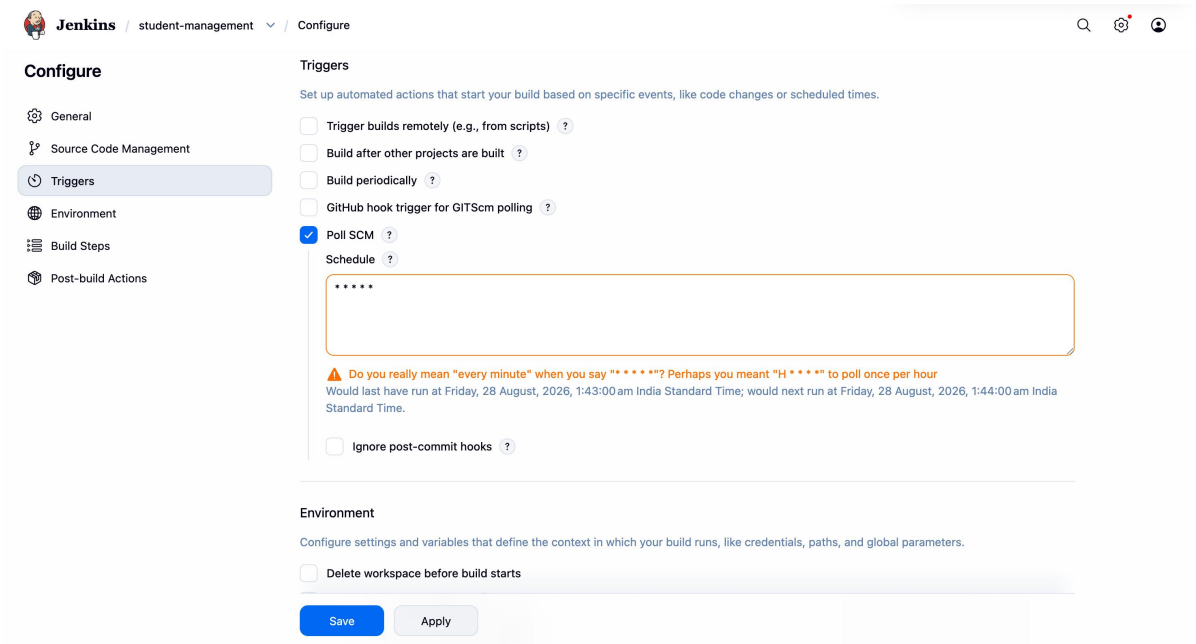
    post {
        success { echo 'SUCCESS - build completed' }
        failure { echo 'FAILURE - check the console output above' }
    }
}
```

The console output of the build shows the repository being cloned, the revision being checked out with the commit message of the merged pull request, each required file being verified, and the build finishing with SUCCESS.

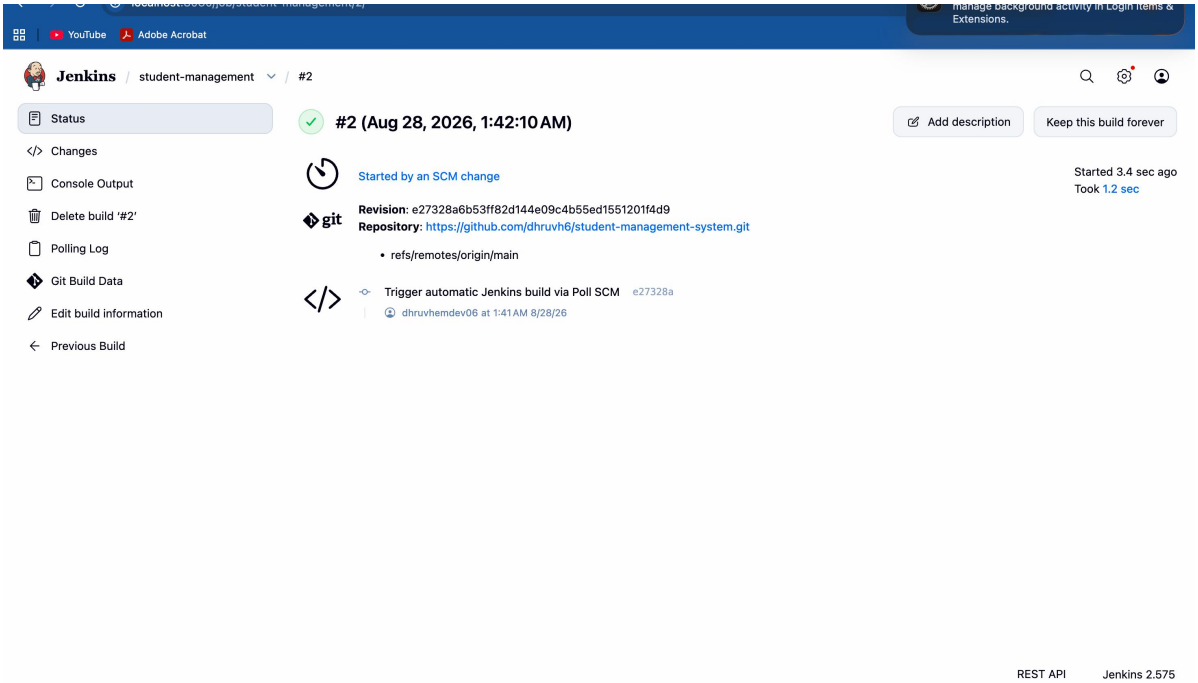


Step 12: Add Automatic Build

So that a build no longer has to be started by hand, an automatic trigger was configured. Because Jenkins is running on localhost it cannot receive an incoming webhook from GitHub, so Poll SCM was used instead. Jenkins checks the repository on the configured schedule and starts a build whenever a new commit is found.

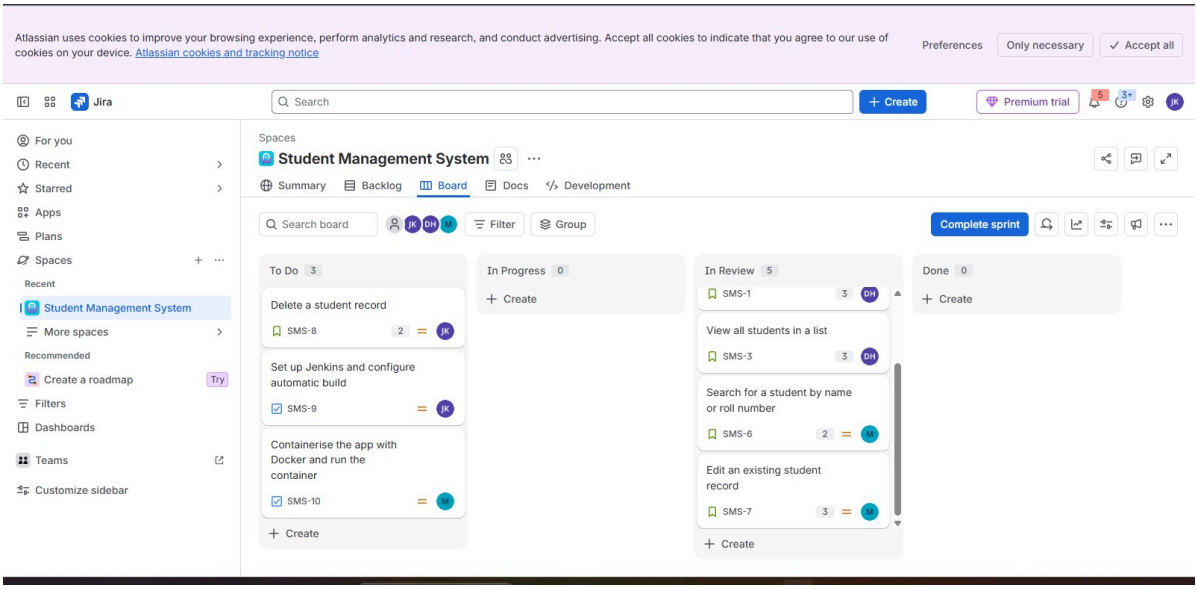


A commit was then pushed to the repository without opening Jenkins. The next build started on its own, and its status page records that it was started by an SCM change together with the revision and the commit message that caused it. This confirms that no manual build is needed after this step.



Step 13: Move the Stories in Jira

As each story was completed it was moved through the workflow on the board, from To Do to In Progress when development started, to In Review when the pull request was raised for a teammate to review, and to Done after the review, the merge and a successful Jenkins build. The board below was taken during the sprint, with several stories already awaiting review and the remaining technical tasks still to be started.

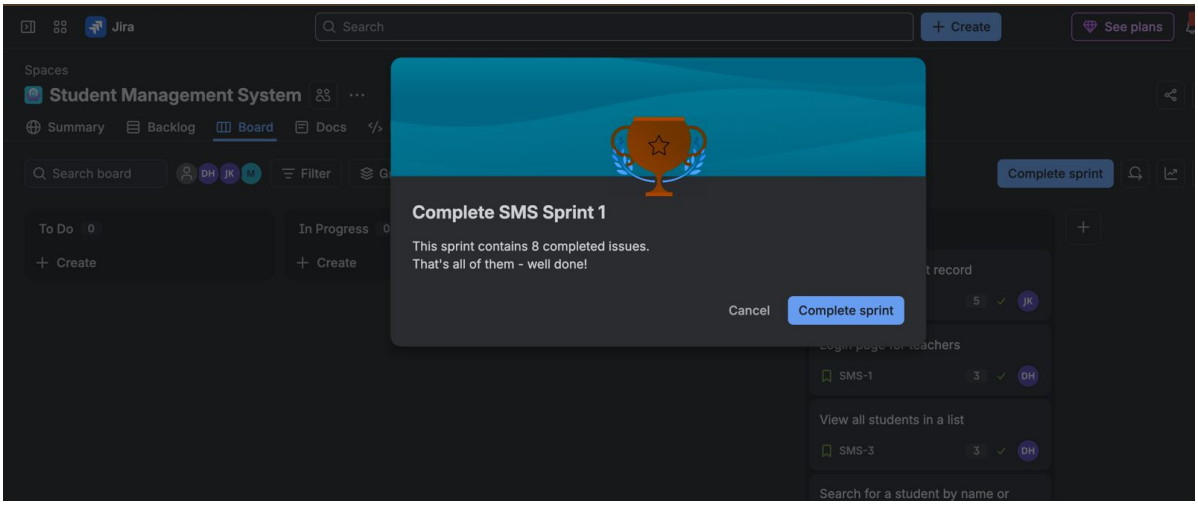


Definition of Done agreed by the team:

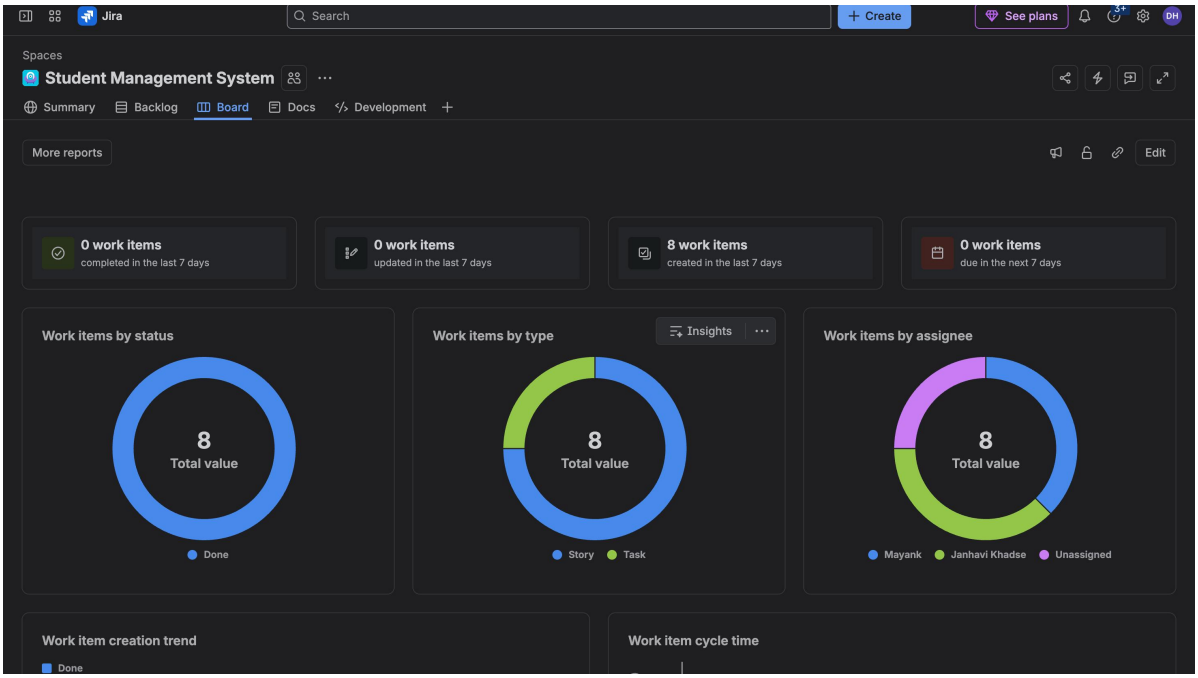
An issue was moved to Done only when the commit message began with the issue key, the pull request had been reviewed by a teammate and merged into main, the Jenkins build of main reported SUCCESS, and the feature had been verified in the running container. The third condition is what ties the board to the pipeline configured in Steps 10 to 12.

Step 14: Complete the Sprint

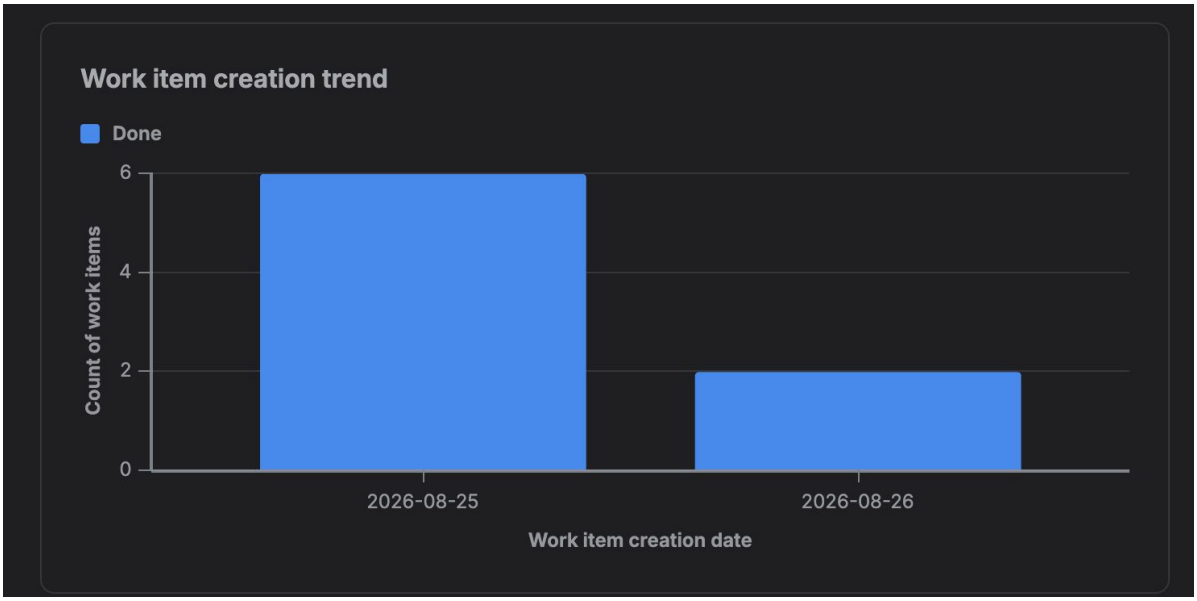
When every issue of the sprint had reached Done, the sprint was closed with Complete sprint. Jira confirmed that the sprint contained eight completed issues and that none were left outstanding, so no work had to be carried forward into a following sprint.



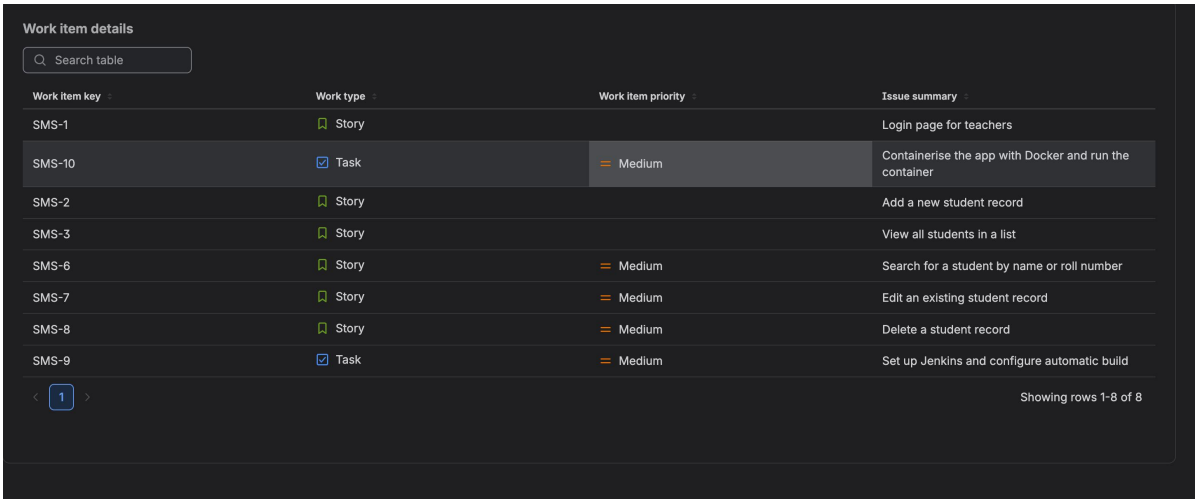
Closing the sprint allows Jira to produce reports from the data recorded during the iteration. The summary shows the eight work items grouped by status, by type and by assignee, which gives the distribution of the completed work across the team.



The creation trend shows when the issues of the sprint were raised, with six created on the first day of planning and the two technical tasks added on the following day.



The work item details report lists every issue of the sprint with its type, priority and summary, and serves as the record of what the sprint delivered.



Step 15: Docker Deployment

Docker was installed and a Dockerfile was added to the repository. The image is based on the official nginx image and copies the application files into the directory from which nginx serves content, so the resulting container serves the application without any further configuration.

```
FROM nginx:latest
COPY . /usr/share/nginx/html
```

The image was then built and run as a container. Port 8081 of the host is mapped to port 80 of the container, because port 8080 on this machine is already occupied by Jenkins.

```
docker build -t student-management .
docker run -p 8081:80 student-management

http://localhost:8081
```

Opening that address in a browser shows the Student Management System running inside the container, which completes the lifecycle from a requirement recorded in Jira to a deployed application.

CONCLUSION :

The experiment successfully demonstrated the Agile software project lifecycle using Jira integrated with a DevOps pipeline. A Scrum project was created in Jira, the requirements of the Student Management System were raised as six user stories and two technical tasks, the stories were estimated at eighteen points in total and divided between the three team members, and all the issues were pulled into SMS Sprint 1 and started.

The application was written as a static web application, stored in a GitHub repository, and developed one story at a time on branches named after their issue keys, with every commit message beginning with the key of the issue it implemented. A Jenkins job was connected to the repository and the pipeline verified the required files and built the container image. Configuring the Poll SCM trigger removed the need to start builds by hand, and a later build was recorded by Jenkins as having been started by an SCM change.

Each story was moved through To Do, In Progress, In Review and Done on the board, and an issue was allowed to reach Done only after its pull request had been reviewed and merged and the Jenkins build of main had reported SUCCESS. The sprint was then completed with all eight issues finished, and Jira generated reports showing the work items by status, type and assignee, the creation trend and the full issue list. Finally the application was containerised with Docker and served by nginx from the running container.

Overall, the experiment provided practical knowledge of Scrum planning, user stories and story point estimation, board and sprint management in Jira, branch and pull request workflow in Git and GitHub, automated builds in Jenkins, container deployment with Docker, and the traceability that results from linking an Agile project lifecycle to an automated DevOps pipeline.